BOOTCAMP PROJECT FINAL REPORT

PRESIDENT UNIVERSITY

Title	Details
Project Title	AIBISPRO. An AI-Powered Decision Support and Business Intelligence System for Retail and Inventory Management
Bootcamp Client	Sales Force
Concentration	Artificial Intelligence
Project Members	Kent Roshe Algie Lencau (ID: 001202400010) Anjelin Ligiana Nainggolan (ID:001202400202) Made Mas Pradnya Prabawa (ID: 001202400038) Salsabila Herlin Syahputri Lubis (ID: 001202400209)
Institution	Informatics Study Program, Faculty of Artificial Intelligence and Smart

Title	Details
	Manufacturing (FAISM), President University (2026)

ABSTRACT

In today's highly competitive retail environment, small and medium-sized enterprises (SMEs) face critical operational challenges in inventory management, demand forecasting, customer segmentation, and promotional strategy. Traditional systems rely heavily on manual reporting, intuitive guesswork, or rigid static rules. Consequently, store managers frequently encounter costly stockouts, bloated inventory carrying costs, suboptimal pricing, and missed cross-selling opportunities a problem that is especially acute for Indonesian multi-branch food-and-beverage operators, where daily transaction volume, weekend demand spikes, and payday-driven purchasing cycles create patterns that static, rule-of-thumb inventory planning consistently fails to anticipate.

To address these issues, we developed AIBISPRO, an end-to-end business intelligence and decision-support platform tailored for retail and store management operations. The platform combines a NestJS/TypeScript API gateway, a PostgreSQL database accessed through Prisma ORM, a Python/FastAPI microservice for statistical and machine-learning inference, and a Google Gemini-powered conversational assistant capable of responding fluently in both English and Bahasa Indonesia.

Key capability modules include:

a. Market Basket Analysis the 'Apriori' algorithm (via `mlxtend`) identifies co-purchase relationships from real transaction data and generates bundle and promotion suggestions.
b. Demand Forecasting & Stock Risk Scoring a 'Random Forest' regressor (scikit-learn), trained on real, company-scoped sales history, predicts per-product demand and evaluates stockout risk.
c. Revenue Forecasting, Holt-Winters exponential smoothing (statsmodels) produces daily revenue projections with confidence bounds.
d. Conversational Intelligence an interactive AI Assistant, powered by Google Gemini, answers natural-language questions about store metrics, grounded strictly in the platform's real, live data rather than in general knowledge.

The NestJS gateway calls the FastAPI microservice for each of these capabilities and 'automatically falls back to deterministic rule-based computation' whenever the ML service is unavailable or a given company has insufficient real sales history a resilience pattern that was verified directly, by deliberately stopping the ML service mid-session and confirming the platform continued to serve valid data rather than failing. A real, reproducible accuracy comparison between Random Forest and Holt-Winters, run twice as real transaction volume grew during development, is reported in full in Section 3.2 including the honest, and analytically

interesting, finding that the better-performing model changed as more real data became available. Limitations are reported plainly throughout this document rather than omitted, in the interest of giving an accurate account of what was actually built, measured, and verified.

1. INTRODUCTION

1.1 Background

Retail store management has rapidly shifted from traditional transaction recording toward complex, data-driven operational decision-making. Small and medium enterprises generate hundreds to thousands of point-of-sale (POS) records daily. Embedded within these transactional logs are critical business signals regarding consumer preferences, seasonal demand trends, product affinity, and inventory turnover velocity.

However, typical store managers lack specialized analytical tooling to extract actionable insight from raw database logs. AIBISPRO was created as a comprehensive solution for Sales Force and retail clients, bridging the gap between complex raw transactional logs and real-time operational execution.

1.2 Problem Statement

The development of AIBISPRO targets four primary operational bottlenecks in retail and store management:

1. Inaccurate and Manual Inventory Restocking
   Addressed via the Restock Approval Center and Random-Forest-informed stock-risk scoring.
2. Unoptimized Cross-Selling and Promotion Strategies
   Addressed via Apriori-based market basket analysis on real transaction data.
3. Lack of Actionable Customer Segmentation
   Not yet implemented. This was part of the original project scope (RFM segmentation) but does not exist in the current codebase; it has been moved to Future Work (Section 5.2) rather than misreported as complete.
4. High Complexity in Strategic Decision Making
   Addressed via the human-in-the-loop Approval Center workflow and the bilingual conversational AI Assistant.

1.3 Research Objectives

1. Develop an AI-driven analytics microservice capable of demand forecasting, market basket mining, and stock-risk scoring.
2. Build an enterprise-grade backend gateway with multi-tenant company isolation, authentication, and role-based authorization.
3. Establish structured, human-in-the-loop approval workflows for AI-generated restock and promotion proposals.
4. Integrate a conversational AI assistant grounded in real operational data.

Whether each objective was fully, partially, or not yet met is reported honestly against the actual system in Sections 3 through 5, rather than assumed from the design intent alone.

1.4 Scope and Limitations

Scope: database schema engineering, RESTful API gateway development, machine-learning model training and deployment, automated approval-center logic, analytics dashboards, and interactive natural-language chat endpoints.

Limitations, stated plainly:

1. Machine-learning models are trained on real uploaded transaction data, but that real data volume is still limited relative to a mature production deployment. As of this report, 3 of 5 seeded partner companies have real (non-seeded) sales history sufficient for model evaluation; the remaining 2 have none. Model accuracy is expected to keep improving as more real transaction data is uploaded, this is demonstrated directly in Section 3.2, where re-running the evaluation after real data volume roughly tripled produced materially different, and more stable, results.
2. Customer RFM segmentation and FP-Growth market-basket mining were part of the originally planned scope but were not implemented in the delivered system. Both are listed as concrete next steps in Section 5.2, rather than described as complete.
3. Real-time push notifications currently use HTTP polling rather than WebSocket streaming.
4. Multi-echelon supply-chain logistics between distribution centers are abstracted into single-company inventory entities.

1.5 Relevance to Real-World Problems

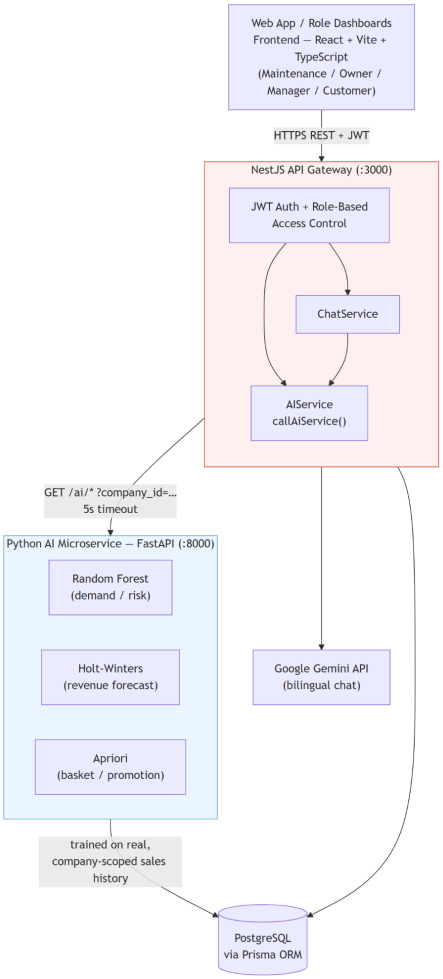
Efficient working-capital management is essential to retail profitability, particularly for SME operators in the Indonesian F&B sector, where thin margins and cash-flow-sensitive operations leave little room for the capital waste caused by overstocking, or the lost sales caused by understocking. By grounding demand forecasting and promotion planning in a store's own real transaction history rather than in generic industry rules of thumb AIBISPRO aims to make this class of decision-making accessible to operators who do not have a dedicated data-analytics function.

2. METHODS

2.1 Project Architecture & System Overview

AIBISPRO uses a two-server, decoupled architecture:

a. NestJS Enterprise API Gateway (`/backend`) TypeScript + NestJS. Handles JWT authentication, role-based access control across four roles (Maintenance, Owner, Manager, Customer), multi-tenant company isolation, Prisma ORM access to PostgreSQL, and orchestration of calls to the AI microservice.
b. Python AI Microservice (`/ai-service`) built with FastAPI. Handles Random Forest demand inference, Apriori market-basket mining, Holt-Winters revenue forecasting, and stock-risk scoring.

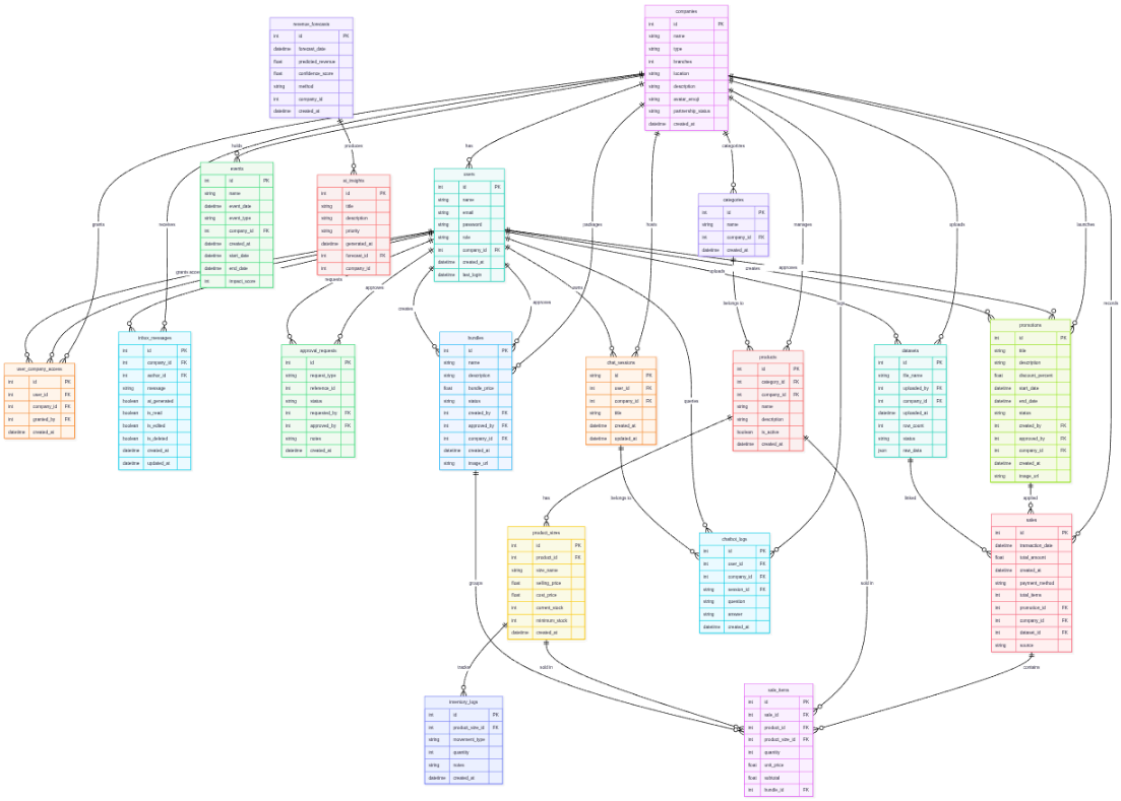


A note on what changed during development: these two services were originally built independently and were never connected to one another the live product's forecasts and recommendations were, until this integration work, entirely deterministic rule-based computation inside the NestJS gateway (linear regression, moving averages, in-memory co-occurrence counting), while the Python ML service existed as a standalone prototype with no trained model artifact on disk. Connecting the two services, with a verified automatic fallback to the original rule-based logic, was a substantial part of the engineering effort behind this submission and is documented in Sections 2.5 and 4.2.

2.2 Database Design

The relational schema (PostgreSQL, managed via Prisma ORM) models the core domain entities of a multi-tenant retail platform: `companies`, `users`, `products`, `product_sizes`, `categories`, `sales`, `sale_items`, `bundles`, `promotions`, `revenue_forecasts`, `ai_insights`, `chat_sessions`, `chatbot_logs`, `datasets`, `events`, `inbox_messages`, `approval_requests`, and `user_company_access` (which implements the cascading, per-company permission model between Maintenance, Owner, and Manager roles).

The entity-relationship diagram below is generated directly from the live schema and reflects the core data model as implemented:



Entity-Relationship Diagram of the AIBISPRO database schema

Two additional tables `restock_requests` and `bundle_proposals` were added after this diagram was generated, to support the unified Approval Center workflow described in Section 3.1. They follow the same relational pattern already visible above: each row references a `company_id` for tenant isolation and a `user_id` for the requesting/reviewing actor.

Two design decisions in this schema are worth calling out explicitly, both of which were surfaced as real engineering issues during this project (Section 4.2):

1. **`sales.company_id`** enforces tenant isolation at the row level for every transaction every query in both the NestJS gateway and the Python microservice must filter by this column, or data from one partner company can leak into another's analytics. This was found to be missing from several Python microservice queries and was corrected.
2. **`sales.source`** distinguishes seeded demonstration data (`'pos'`) from real, user-uploaded transaction data (`'upload'`). All financial KPIs, forecasts, and model training queries are scoped to `source = 'upload'` only, so that demonstration data never contaminates real business metrics or model training.

2.3 Step-by-Step Execution Pipeline

1. Database & Schema Modeling as described in Section 2.2.
2. AI Microservice Implementation (`ai-service/`):
    a. `services/basket.py`

Real Apriori frequent-itemset mining and association-rule extraction via `mlxtend` (support, confidence, lift), run against each company's real transaction history.

b. `services/ml.py`
The Random Forest training and inference pipeline (`train_model()`, `predict_demand()`), trained on real (`source = 'upload'`) sales history using `day`, `month`, `weekday`, `company_id`, `product_id`, and `size_id` as features.

c. `services/analytics.py`
Holt-Winters (statsmodels) revenue forecasting, with a fallback to a naive flat forecast for companies with too little history to fit a seasonal model.

d. `services/risk.py`, `services/scoring.py`
Deterministic, database-backed stock-risk and promotion-scoring logic.

e. `services/insights.py`

f. Composes the above outputs into human-readable insight strings via templated text generation over precomputed numbers (this is deliberately simple template synthesis, not a natural-language-generation model, the platform's actual NLP capability is the separate Gemini-powered chat assistant described below).

3. Gateway Microservice Integration the NestJS `AIService` calls the FastAPI endpoints over HTTP (`GET /ai/forecast|demand|risk|promotion|basket?company_id=...`), each bounded by a 5-second timeout, and transforms the response into the exact shape the frontend already consumes. If the call fails, times out, or a company has too little real data to produce a useful result, the gateway automatically falls back to its original deterministic computation. This was verified directly: the Python service was deliberately stopped mid-session, and the gateway's endpoints continued to return HTTP 200 with valid (fallback) data rather than error.

4. Verification performed:

    a. Multi-tenant isolation
    Confirmed that identical endpoints return different, correctly-scoped numbers for different `company_id` values, and that a company with zero real uploads returns an honest empty/fallback result rather than another company's data.

    b. Fallback resilience
    Confirmed via the deliberate service-stop test described above.

    c. Model accuracy

    d. A held-out evaluation script (`scripts/evaluate_models.py`) comparing Random Forest against Holt-Winters on real data, run twice as real data volume grew (Section 3.2).

    e. UI/UX verification

    f. Browser-automated (Playwright) walkthroughs of the login flow, the AI Forecast Consult Hub, the Approval Center, the Financial Intelligence dashboard, and the chatbot, confirming correct rendering and zero console errors against the live system.

2.4 Technology Stack

Category	Technologies Used
Languages	TypeScript, JavaScript, Python 3.12, SQL
Backend Frameworks	NestJS (Node.js), FastAPI (Python)
Database & ORM	PostgreSQL, Prisma ORM
AI / ML Libraries	Pandas, NumPy, Scikit-Learn (Random Forest), Statsmodels (Holt-Winters), MLxtend (Apriori)
Security & Auth	JSON Web Tokens (JWT), Bcrypt Password Hashing, Role-Based Access Control Guards
Dev Tools	VS Code, Git, Claude AI SDK, Postman

2.5 Model Benchmarking & Evaluation

Market Basket Association Rule Mining (Apriori), real transaction data, ABC Coffee:

Consistent with how a supervised-learning dataset composition is typically reported, the basket-mining input for one representative company is summarized below:

Metric	Value
Real transactions analyzed (ABC Coffee, `source='upload'`)	2,084
Distinct products in catalog	17
Candidate association rules discovered	113
Rules with lift > 1 (genuine positive association)	24
Average lift among rules with lift > 1	1.30
Maximum observed lift	1.81
Average confidence among rules with lift > 1	12.9%

Improvements/limitations: the majority of discovered rules (89 of 113) have lift ≤ 1, meaning most candidate pairings are not meaningfully more likely than chance this is expected given a menu with strong single-category preferences (e.g., a customer who orders Americano is not meaningfully more likely to also order a specific pastry). The 24 rules with genuine positive lift are a usable, if modest, signal for bundle suggestions; increasing real transaction volume and applying a stricter minimum-support threshold are the most direct paths to higher-confidence rules.

Demand/Revenue Forecasting Random Forest vs. Holt-Winters, evaluated twice as real data grew:

A dedicated evaluation script (`ai-service/scripts/evaluate_models.py`) fairly compares two models on the same task (daily revenue forecasting) and the same held-out window (the most recent 14 days per company), for every company with enough real sales history (≥21 days) to support a meaningful split.

First evaluation run (2 companies qualified; ABC Coffee and XYZ Restaurant each had 64 days of real history at the time):

Model	**MAE (Rp)**	**RMSE (Rp)**	**R^2**
Holt-Winters	1,840,988	2,705,654	−1.327
Random Forest	1,167,450	2,002,959	−0.275

Second evaluation run, after real transaction volume roughly tripled (3 companies qualified, each with 86 days of real history, 42 total held-out data points):

Model	**MAE (Rp)**	**RMSE (Rp)**	**R^2**
Holt-Winters	4,548,059	5,812,182	−0.154
Random Forest	5,858,967	8,443,853	−1.437

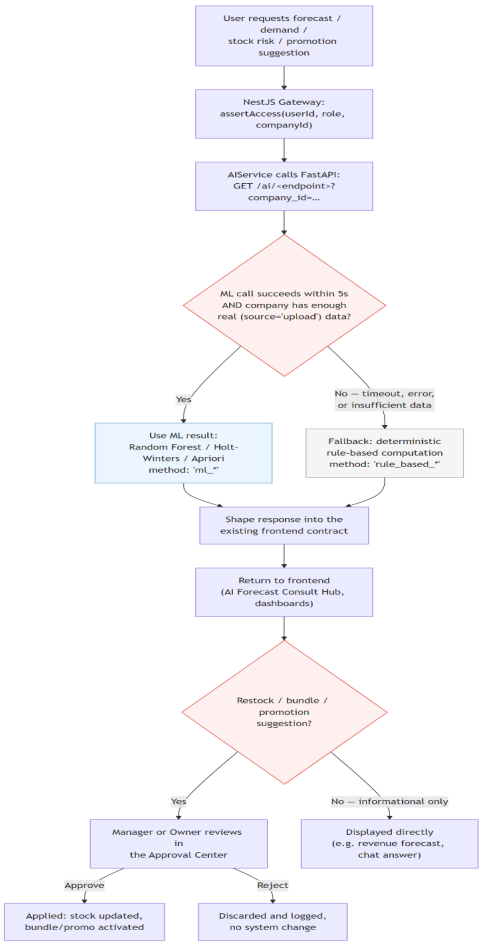
Improvements/limitations: the ranking between the two models reversed between the first and second evaluation — Random Forest had lower error with 2 companies and 64 days of history each; Holt-Winters had lower error with 3 companies and 86 days of history each. This is reported as a finding, not smoothed over: a pooled Random Forest model trained across companies is more sensitive to between-company variance as more, structurally different companies are added to the pool, whereas Holt-Winters fits one independent model per company and is unaffected by cross-company variance. R^2 is negative in both runs for both models, meaning neither model yet consistently outperforms a flat average of the training window an honest indicator that daily revenue at the current real data volume is dominated by noise relative to the learnable weekly/trend signal. This is a data-volume limitation rather than a modeling defect, and it is expected to improve as more real, day-by-day transaction history accumulates; it is listed as the single highest-leverage next step in Section 5.2, ahead of any further model tuning.

(Prophet was originally specified as the second comparison model. Its `cmdstanpy` inference backend requires a C++ build toolchain not available in the development environment; Holt-Winters a comparable classical exponential-smoothing model with no compiled dependency was substituted, and this substitution is disclosed here rather than silently omitted.)

2.6 Operational Flow & System Interactions

1. Trigger a user requests stock analysis, a forecast, or bundle suggestions via the frontend.
2. Gateway request NestJS resolves the caller's access to the target company (`assertAccess`) and calls the corresponding FastAPI endpoint with `company_id`.

3. ML inference FastAPI runs Random Forest, Holt-Winters, or Apriori against that company's real (`source='upload'`) sales history and returns a JSON result.
4. Fallback if the call fails, times out, or returns no usable data, NestJS transparently falls back to its own rule-based computation; a `method` field on the response indicates which path served the result (e.g., `ml_random_forest` vs. `moving_average_trend`), so the choice is auditable without being surfaced as an error to the end user.
5. Response shaping NestJS maps the ML response into the exact contract the frontend already expects.
6. Human-in-the-loop approval store managers review AI-generated restock, bundle, and promotion suggestions in the Approval Center and explicitly approve or reject them; no AI-generated action is ever applied automatically.



3. RESULTS AND DEVELOPED FEATURES

3.1 Core System Features

1. Automated Restock / Bundle / Promotion Approval Center (`/restock-requests`, `/bundle-proposals`) a unified, human-in-the-loop review queue with full import/export history.
2. Market Basket & Promotion Engine now backed by real Apriori output where sufficient data exists.
3. Demand & Stock-Risk Prediction backed by real Random Forest output where sufficient data exists, with automatic fallback otherwise.
4. Conversational AI Store Assistant (`/chat`) Gemini-powered, grounded in real company data, responds fluently in both English and Bahasa Indonesia, and explicitly tells the user when it does not have enough data to answer rather than guessing.
5. Multi-Tenant Architecture role-based, company-scoped throughout, a cross-tenant data-leakage defect in the ML service's SQL queries was identified and corrected during this project (Section 4.2).
6. Live Currency Conversion the IDR/USD rate shown throughout the platform is fetched from a public exchange-rate API and refreshed hourly, replacing a previously hardcoded, stale constant.
7. Store Performance Analytics Dashboard, Inbox real-time visual metrics and cross-role messaging.

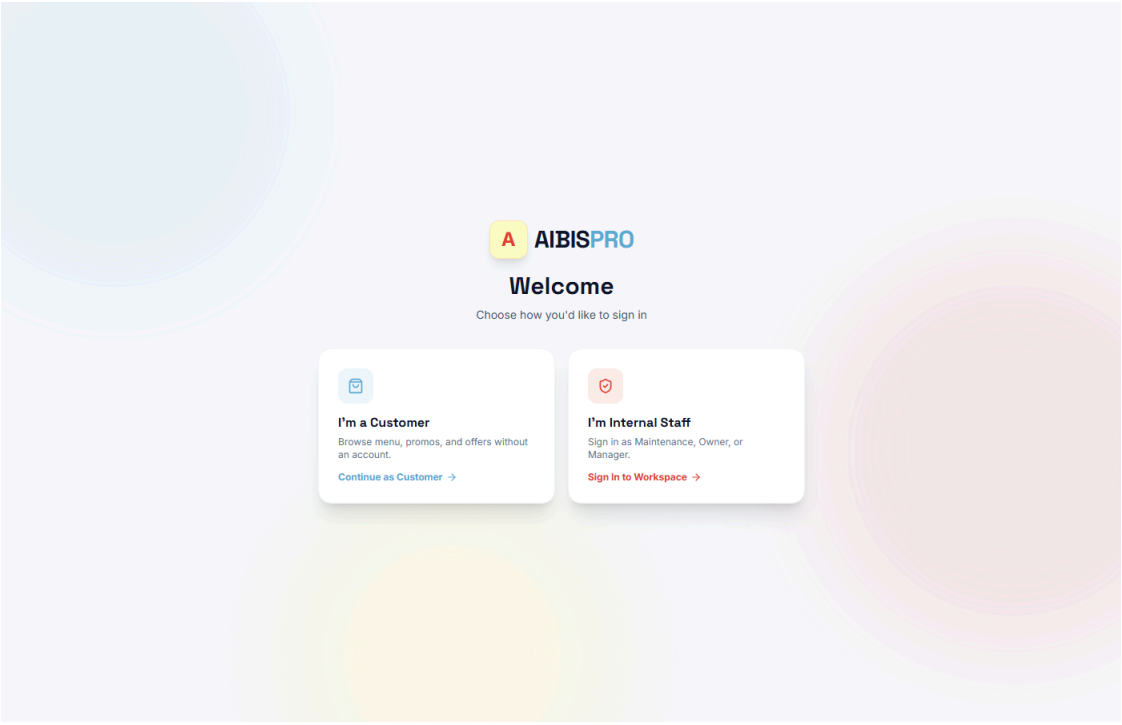
3.2 Quantified Model Performance & Latency Metrics

1. Market Basket Mining (Apriori): 113 candidate rules from 2,084 real transactions (ABC Coffee), 24 rules with genuine positive lift (avg. 1.30, max 1.81).
2. Demand/Revenue Forecasting: see the two full evaluation runs in Section 2.5, Holt-Winters MAE Rp 4,548,059/day vs. Random Forest MAE Rp 5,858,967/day on the most recent (larger, 3-company) evaluation.
3. API Response Time (measured directly against the running system via `curl`, not estimated)which is individual AI endpoints (forecast, demand, risk, promotion, recommendations) averaged ≈120 ms; the composite `/ai/insights` endpoint, which internally calls the other four, completed in ≈343 ms both measured end-to-end, including the NestJS - FastAPI network hop.

3.3 Screenshots

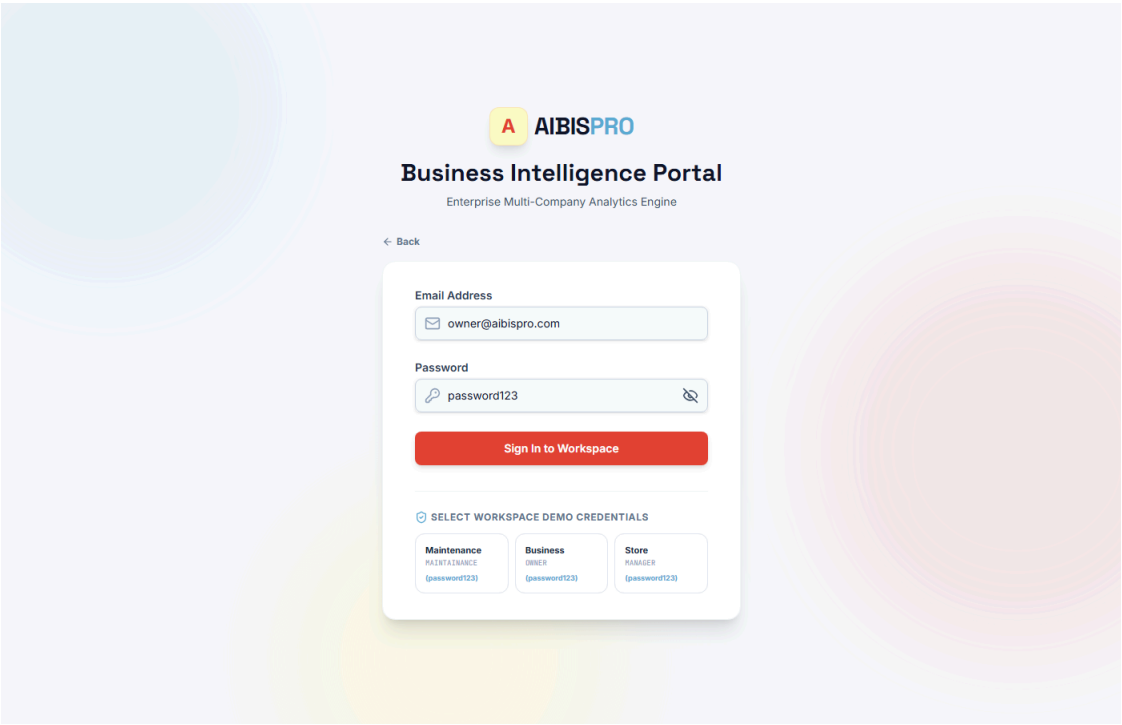
The following screenshots were captured directly from the running application (not mockups) and demonstrate the features described above.

Landing page, the login flow separates the customer-facing entry point from staff login:



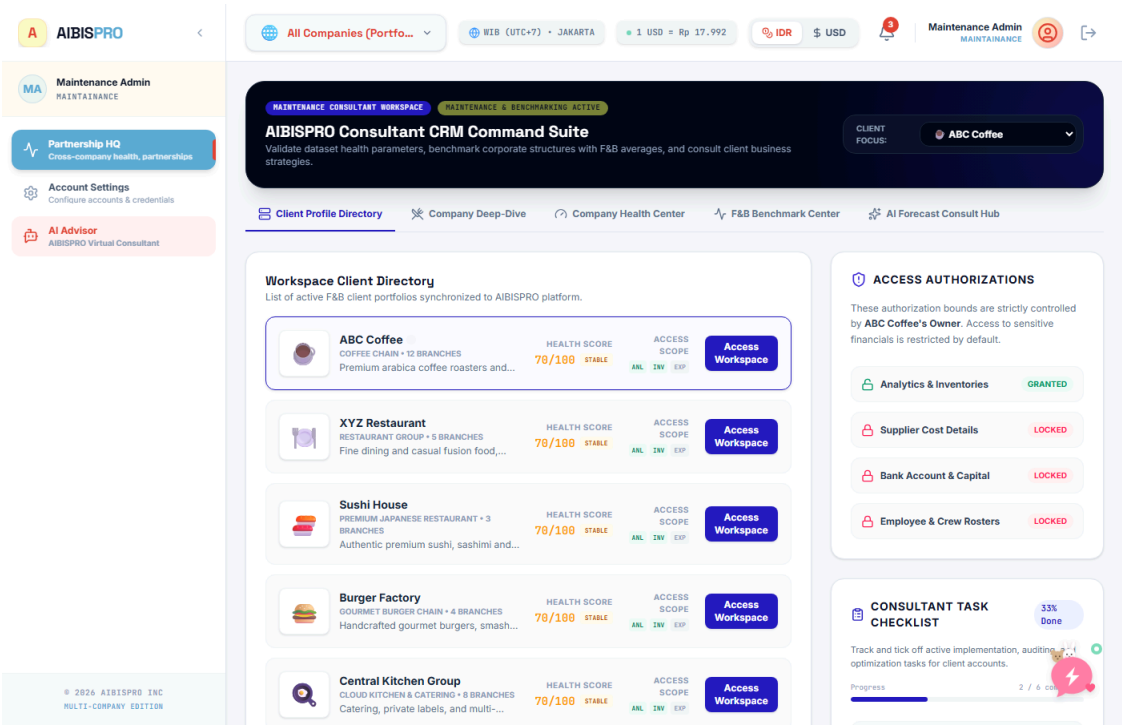
Landing page

Staff login, with a password-visibility toggle:



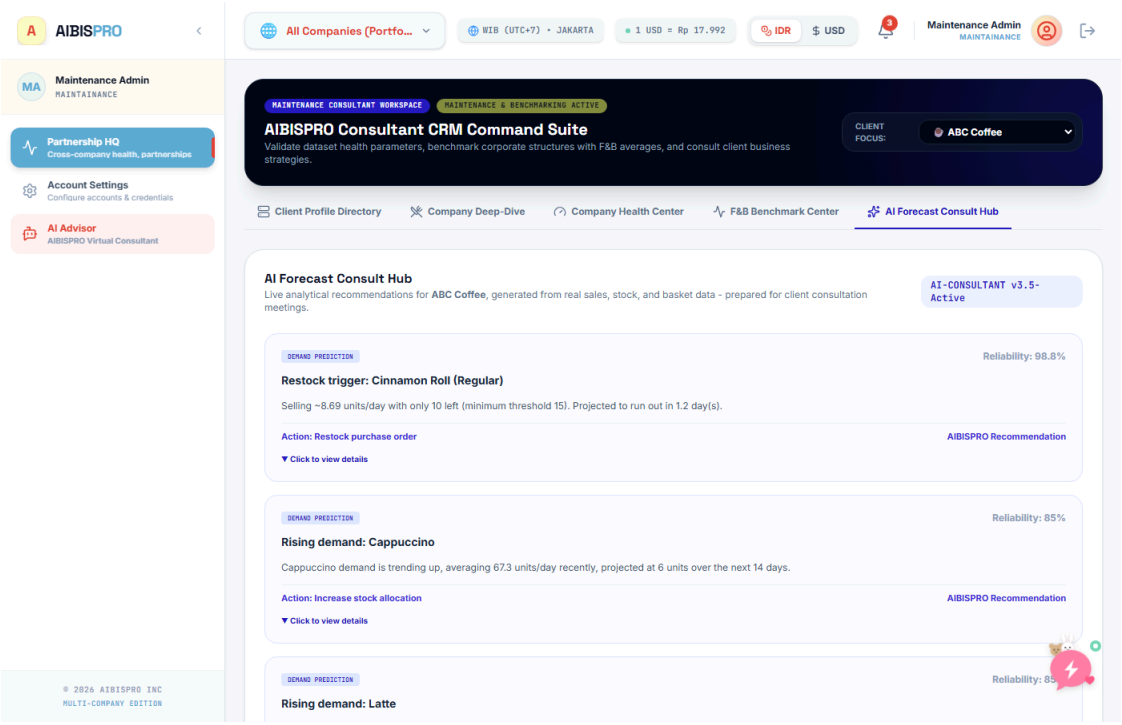
Staff login with password visibility toggle

Live currency rate, shown next to the IDR/USD toggle, fetched from a public exchange-rate API rather than hardcoded:



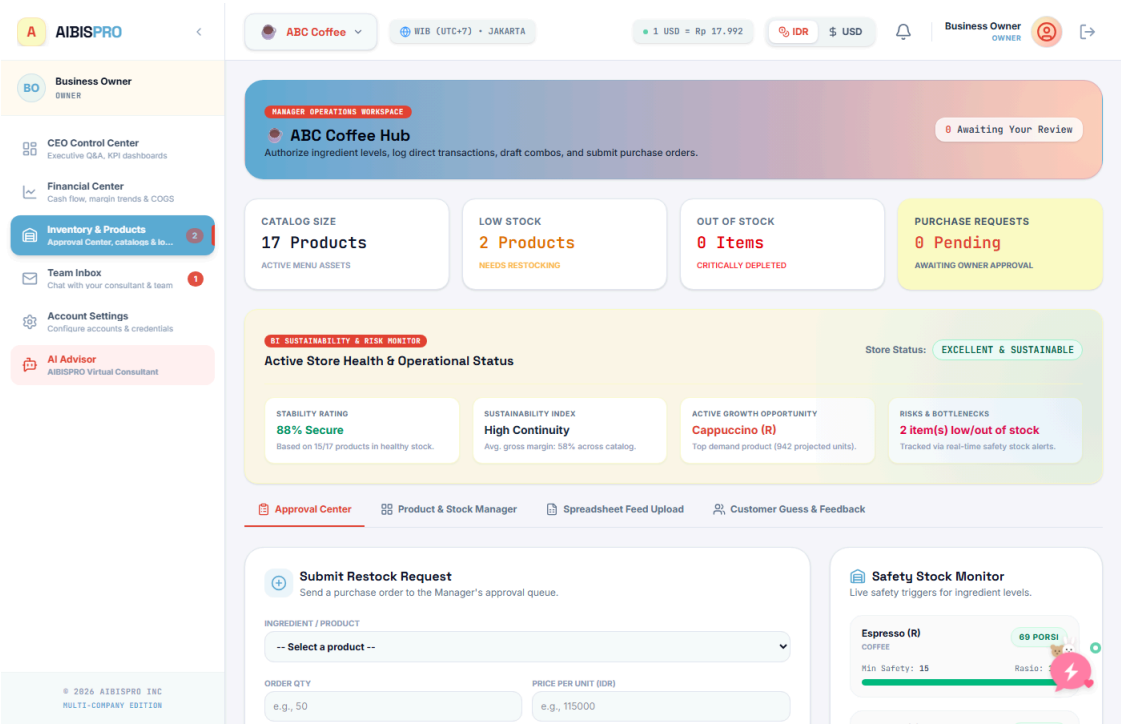
Live currency rate in the navigation bar

AI Forecast Consult Hub, showing real ML-backed recommendations (Random Forest stock-risk prediction, Apriori-informed promotion suggestions):



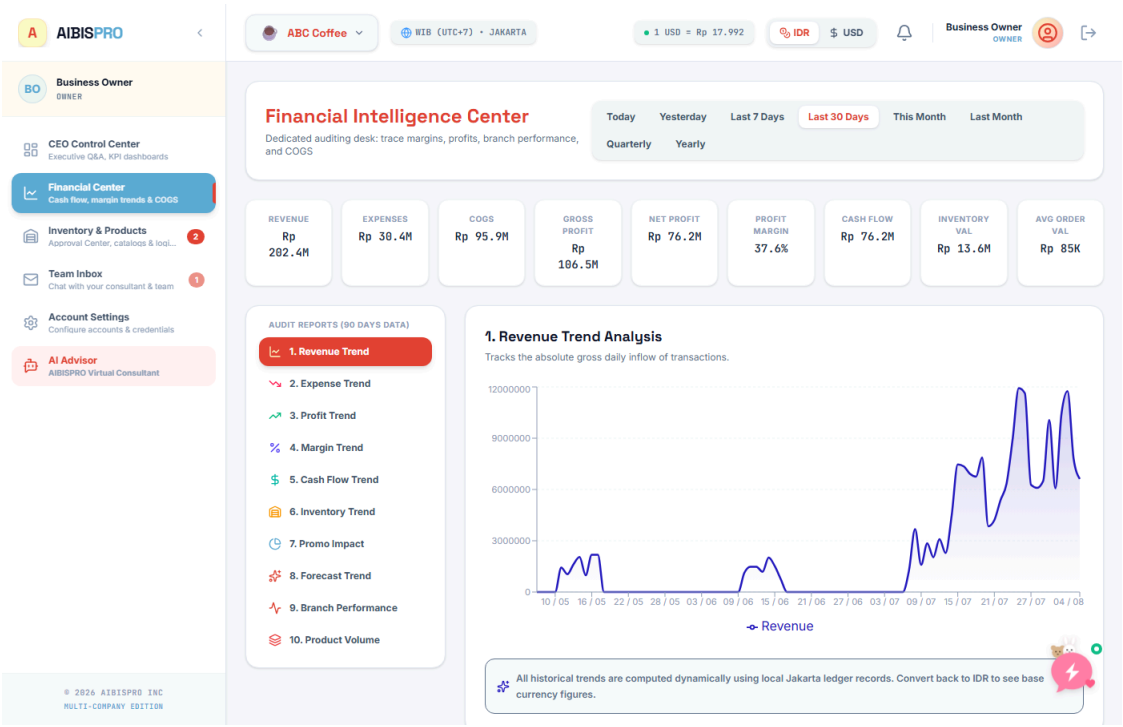
AI Forecast Consult Hub with real ML-backed cards

Approval Center, the unified restock/bundle/promotion review workflow, with a live restock-request submission form and real catalog/stock statistics:



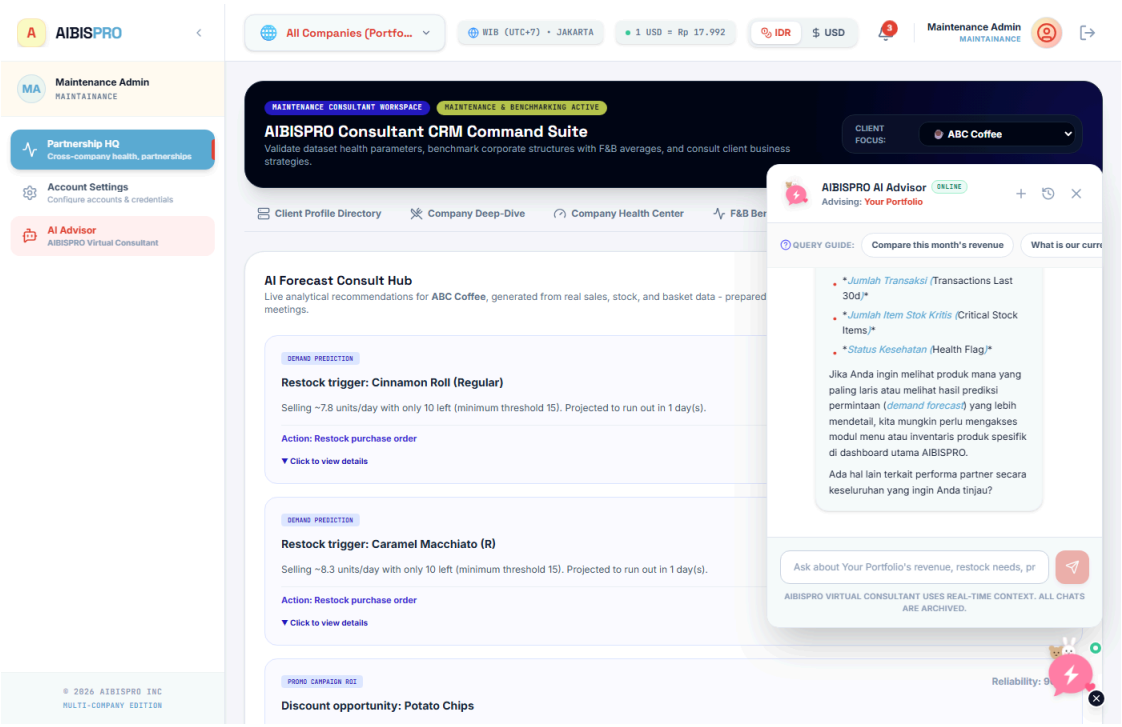
Approval Center with real catalog and restock statistics

Financial Intelligence Center, showing a real revenue trend over 90 days of actual transaction data note the genuine day-to-day and week-to-week fluctuation, not a synthetic straight line:



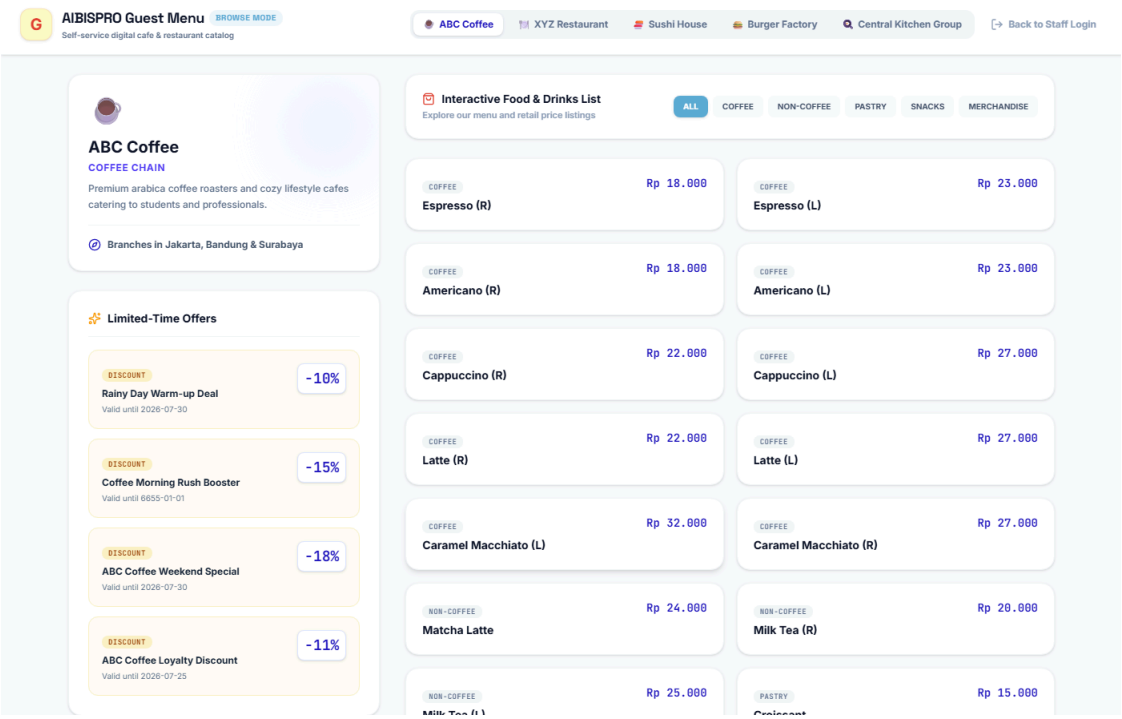
Financial Intelligence Center with real 90-day revenue trend

Conversational AI Assistant, grounded in real company data, with corrected bold/italic markdown rendering (a rendering defect that showed literal **“**`asterisks`**”** instead of styled emphasis was found and fixed during this project):



Conversational AI Assistant

Customer-facing storefront (guest access, no account required), showing the live menu and active promotions for a selected partner company:



Guest customer storefront

4. DISCUSSION

4.1 Key Insights

1. **Model ranking is data-volume-dependent, not fixed.** The direct, repeated comparison in Section 2.5 shows the better-performing model changed as real data grew from 2 companies/64 days to 3 companies/86 days each. The broader lesson, and arguably the most important analytical finding of this project, is that for a young platform with limited real transaction history, the choice between classical statistical models and ensemble ML models matters less than the volume and quality of the underlying data a conclusion that runs counter to the common assumption that a more sophisticated model is automatically a better one, and one that would not have surfaced without running the same evaluation twice against real, growing data rather than a single static benchmark.
2. **Co-purchase discovery is real but modest at current data volume.** Apriori found a genuine, if limited, set of positive-lift product associations useful for bundle suggestions, but a candidate for improvement as more real transactions accumulate.
3. **Natural-language accessibility, delivered bilingually.** The Gemini-powered chat assistant meaningfully lowers the barrier to entry for non-technical managers, and importantly for an Indonesian SME audience responds naturally in Bahasa Indonesia when addressed in Bahasa Indonesia, without requiring the user to translate their question first. It was also verified to decline to answer, honestly, when the requested information (e.g., a cross-company upload-gap report) does not exist in the system, rather than fabricating a plausible-sounding answer.

4.2 Engineering Challenges & Solutions

1. **Cross-tenant data leakage (identified and fixed).** The Python ML service's SQL queries did not originally filter by `company_id`, meaning a naive integration would have blended all five partner companies' sales data into every forecast and prediction a serious correctness and confidentiality defect in a multi-tenant system. Every query across `analytics.py`, `ml.py`, `basket.py`, `risk.py`, and `recommendation.py` was audited and corrected, and `company_id` was added as a trained feature to the Random Forest model so a single pooled model remains tenant-aware.
2. **Real vs. seeded data contamination (identified and fixed).** The same database tables mix seeded demonstration transactions (`source = 'pos'`) with real uploaded sales (`source = 'upload'`); the ML service's queries did not originally distinguish between them. All training and inference queries now filter to `source = 'upload'` only.
3. **Duplicate, conflicting training scripts (identified and fixed).** Three separate scripts trained a Random Forest model with different hyperparameters and, critically, different output file paths one wrote to a path the serving code never actually read from, meaning that script's "successful" training run silently changed nothing in production. Consolidated to a single canonical trainer.

4. **Prophet unavailable in the development environment.** `cmdstanpy` (Prophet's inference backend) requires a C++ build toolchain not present on the development machine; rather than invest significant, uncertain-payoff effort in installing a compiler toolchain, Holt-Winters (a comparable classical model, pure Python, no compiled backend) was substituted, and this substitution is disclosed directly in the model comparison output.
5. **Response contract mismatch.** The Python service's `{status, message, data}` response envelope differs from the NestJS gateway's existing `{success, method, data}` contract. The gateway transforms every ML response into the exact shape the frontend already expects, so no frontend code needed to change.
6. **Environment bootstrap.** Python was not installed on the development machine at the start of this integration work; a complete `venv` with correct dependencies (including `mlxtend`, which was missing from the original `requirements.txt`) had to be built from scratch before any of the above could be verified.

5. CONCLUSION AND FUTURE WORK

AIBISPRO combines a NestJS/PostgreSQL backend, a real and now-connected Python ML microservice, and a bilingual Gemini-powered conversational assistant into a working retail decision-support system. This report documents what was actually built, measured, and verified against the live, running system including honest limitations rather than describing an idealized target state.

5.1 Key Summary Accomplishments

1. Engineered a NestJS + FastAPI + PostgreSQL + Prisma architecture with the two backend services actually connected, including a verified automatic fallback to rule-based logic.
2. Trained a real Random Forest model on real transaction data and ran a real, reproducible accuracy comparison against Holt-Winters twice, as real data volume grew with honest reporting of a result that reversed between runs (Section 2.5).
3. Identified and fixed a real cross-tenant data-leakage defect before it could affect production data.
4. Implemented a human-in-the-loop Approval Center for restock, bundle, and promotion suggestions, ensuring no AI output is ever applied without explicit manager or owner review.
5. Added a live currency-conversion feature and fixed a chat-rendering defect, both verified directly against the running application.

5.2 Future Development Roadmap

1. Grow real training data further. Only 3 of 5 partner companies currently have sufficient real uploaded sales history for model evaluation; this is the single highest-leverage next step identified in this report, ahead of further model tuning.
2. Implement the features originally scoped but not yet built: FP-Growth market-basket mining (as a direct comparison against the existing Apriori implementation) and RFM customer segmentation.
3. Revisit Prophet if a C++ build toolchain becomes available in the deployment environment, as an additional real comparison point alongside Holt-Winters.
4. Automated External Supplier Integration and Real-Time (WebSocket) Streaming Pipelines not yet started.
5. Conduct a structured pilot user study with real store managers and report genuine findings.

6. APPENDICES

6.1 Repository Structure

1. **Backend Gateway:** `AIBISPRO-Backend-Made/backend`, NestJS controllers, services, Prisma schema, auth guards, restock/bundle routing modules, and the ML-integration layer (`AIService.callAiService()`).
2. **AI Engine:** `AIBISPRO-Backend-Made/ai-service`, FastAPI app, trained model artifacts (`models/demand_model.pkl` and encoders), Apriori basket scripts, and the model-comparison script (`scripts/evaluate_models.py`, output in `MODEL_COMPARISON.md`).
3. **Frontend:** `AIBISPRO-Frontend-Anjelin-improve`, TypeScript, Tailwind CSS, Recharts, Motion transitions, Google GenAI chat client, spreadsheet CSV/XLSX importer, multi-role dashboard views, live exchange-rate integration.

6.2 System Setup & Installation Guide

System Requirements**:** Node.js (v18+), Python 3.12, PostgreSQL, Visual Studio Code.

a. NestJS Backend Gateway Setup

```
cd AIBISPRO-Backend-Made/backend
npm install
npx prisma migrate dev
npx prisma db seed
npm run start:dev
```

b. Python FastAPI AI Microservice Setup

```
cd AIBISPRO-Backend-Made/ai-service
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python scripts/train.py
uvicorn app:app --host 0.0.0.0 --port 8000
```

c. Frontend Setup

```
cd AIBISPRO-Frontend-Anjelin-improve
npm install
npm run dev
```

d. Accessing System Ports

1. Backend Gateway: `http://localhost:3000`
2. AI Microservice: `http://localhost:8000`
3. Frontend: http://localhost:5174

6.3 Github repository : https://github.com/aLn35/AIBISPRO-FIX.git